Documentație Proiect: Exemple de atacuri

Autor: Cosma Andrei 1342a | **Obiectiv:** Demonstrarea exploatării și a prevenirii atacurilor SQL Injection, Phishing și Brute Force.

I. Atacul SQL Injection (SQLi)

SQL Injection este o vulnerabilitate critică (clasificată ca **OWASP A3: Injection** în standardele globale) cauzată de încrederea aplicației în datele introduse de utilizator.

1. Vectorul de Atac: Concatenarea Directă (Endpoint Vulnerabil)

Vulnerabilitatea apare când input-ul utilizatorului este lipit direct în interogarea SQL, prin intermediul metodei `FromSqlRaw`.

- **Payload Demo:** `admin' OR '1'='1 --` (Bypass de autentificare)
- **Cod Vulnerabil (C#):**

```
public async Task<User?> GetUserAsync(string username, string password)
{
    var query = $"SELECT * FROM Users WHERE Username = '{username}' and
Password = '{password}'";

    return await _context.Users
        .FromSqlRaw(query)
        .FirstOrDefaultAsync();
}
```

2. Soluția: Programare Defensivă (Endpoint Securizat)

Prevenirea SQLi se realizează prin utilizarea Interogărilor Parametrizate (sau LINQ, care le folosește nativ) și, adițional, prin hashing-ul parolelor.

- **Mecanismul de Apărare:** Utilizarea **LINQ** (`FirstOrDefaultAsync`) garantează că input-ul pentru `username` este tratat ca **dată literală**, nu ca logică SQL. În plus, parolele sunt verificate folosind **BCrypt** (hashing robust).

- **Cod Securizat (C#):**

```
public async Task<User?> GetUserSecuredAsync(string username, string
password)
{
    var user = await _context.Users
        .FirstOrDefaultAsync(u => u.Username == username);

    if (user == null) return null;

    bool isPasswordValid = BCrypt.Net.BCrypt.Verify(password,
user.PasswordHash);

    if (!isPasswordValid) return null;
    return user;
}
```

Puncte Cheie de Securitate Adresate în Codul Securizat:

1. **Prevenirea SQLi:** Prin utilizarea LINQ în locul string-urilor brute.
2. **Protecția Parolei:** Prin verificarea hash-ului (BCrypt) în locul comparației directe a parolei.

Prevenirea SQLi se realizează exclusiv prin separarea logicii de date.

- **Mecanismul de Apărare: Interogările Parametrizate (Prepared Statements)** tratează întotdeauna input-ul utilizatorului ca **date literale (șir de caractere)**, indiferent de conținutul acestuia.

- **Cod Securizat (Exemplu):**

```
// Interogarea este definită static, separat de date.
QUERY = "SELECT * FROM users WHERE username = ? AND password = ?;"
// Datele sunt transmise ca parametri separați.
db.execute(QUERY, [user_input, pass_input]);
```

II. Atacul de tip Phishing

Atacul demonstrează furtul de credențiale printr-un e-mail fals care direcționează victima către un site de *spoofing*.

1. Furtul de Credențiale și Detecția

Backend-ul salvează datele pentru a simula analiza post-incident.

Data	IP Sursă	Username	Parolă
[Data/Ora]	[IP Sursă]	victima@exemplu.com	parola_furata!

- **Logare:** Pe lângă username și parolă, salvarea **adresei IP** și a **momentului exact** al capturii este vitală pentru detecția și blocarea sursei atacului (simulare SIEM simplificat).

2. Apărarea Esențială: MFA

- **Mecanismul:** Cel mai robust strat de apărare împotriva furtului de parolă este **Autentificarea Multi-Factor (MFA)**. Chiar dacă parola este compromisă prin phishing, atacul eșuează fără al doilea factor de verificare (cod SMS, aplicație, token).

✉ Prevenția la Nivelul E-mailului (Standarde)

Aceste protocoale ajută la verificarea autenticității expeditorului și la blocarea *spoofing*-ului adresei:

- **SPF (Sender Policy Framework):** Verifică dacă adresa IP de trimitere este autorizată să trimită e-mailuri pentru domeniul respectiv.
- **DKIM (DomainKeys Identified Mail):** Adaugă o semnătură digitală e-mailului, asigurând că mesajul nu a fost modificat în tranzit.
- **DMARC (Domain-based Message Authentication, Reporting, and Conformance):** Oferă o politică pentru modul în care serverele receptoare trebuie să trateze e-mailurile care eșuează verificările SPF și DKIM (ex: le respinge sau le trimite în spam).

🧍 Prevenția la Nivel de Utilizator

Instruirea utilizatorilor să verifice două elemente critice înainte de a da click:

1. **Adresa de E-mail a Expeditorului:** Verificarea atentă a domeniului (ex: `micros0ft.com` sau `rnicrosoft.com` în loc de `microsoft.com`).
2. **URL-ul Linkului:** Trecerea cursorului peste link pentru a vizualiza destinația reală, care ar trebui să corespundă domeniului așteptat.

III. Apărarea la Nivel de Infrastructură (Rate Limiting)

Această secțiune demonstrează protecția împotriva atacurilor de forță brută (Brute Force) care vizează epuizarea resurselor și spargerea parolelor prin încercări repetate.

1. Implementare și Mecanism

- **Funcție:** Un Rate Limiter urmărește cererile de login eșuate, asociind contorul cu adresa IP a clientului.
- **Regula Aplicată:** De exemplu: *Permite max. 5 încercări eșuate / 60 de secunde.*

2. Rezultatul Defensiv

- **Răspunsul Serverului:** La depășirea limitei, backend-ul blochează cererile și returnează codul de stare `HTTP 429 Too Many Requests`.
- **Impact:** Acest mecanism protejează disponibilitatea serverului și face ca atacul Brute Force să fie ineficient din punct de vedere temporal.

💡 Concluzie

Proiectul demonstrează necesitatea unei abordări stratificate, cunoscută sub numele de **Defense in Depth**. Nicio vulnerabilitate nu poate fi adresată de o singură măsură. Succesul în securitatea aplicațiilor depinde de:

1. **Programare Sigură:** Utilizarea **Interogărilor Parametrizate**.
2. **Educație:** Implementarea și promovarea **MFA**.
3. **Monitorizare: Rate Limiting** și analiza log-urilor.